Network Anomaly Detection Platform

NetGuard — Formal Summative Project Specification

Status	Project	Team size	Duration
FINAL	Network Anomaly Detection Platform (NetGuard)	5 students	10 teaching weeks

1. Main Project Goal

The goal of the project is to design and implement a distributed platform for monitoring network traffic and automatically detecting and classifying anomalies. The system shall support an operator in identifying unusual behaviour, provide evidence explaining each detection, and enable historical analysis.

The project is strictly defensive. It must not be used to perform real attacks, scan third-party networks, or collect data from production systems. All demonstrations and tests shall use prepared PCAP/flow datasets and a controlled network-traffic simulator.

2. Context and Problem

Modern IT environments are exposed to events that may manifest themselves as unusual network-traffic patterns. In a cyber-defence environment, it is not sufficient to detect a single event: the system should analyse behaviour over time, reduce false alarms, and provide an operator with evidence explaining why an alert was generated.

The project models this problem in a safe laboratory environment and combines stream processing, detection algorithms, database persistence, APIs, dashboards, performance testing, and resilience.

3. Users and Stakeholders

- Security operator / administrator — monitors traffic and alerts.
- Analyst — investigates historical events and detection quality.
- System administrator — configures data sources and detection parameters.
- Development team — implements, integrates, tests, and documents the system.
- Module instructor — evaluates architecture, algorithms, testing, documentation, and system operation.

4. Scope and Implementation Rules

Every team shall implement exactly the same system and mandatory scope. Replacing the platform with a simple CRUD application, an isolated ML model, or an isolated dashboard is not acceptable.

The team may choose programming languages, frameworks, databases, message brokers, and libraries. The choice is acceptable only if all requirements are met and the technology decisions are justified in the Architecture Document.

The complete system must be reproducibly deployable. Containerisation is recommended. The repository must contain installation, configuration, deployment, testing, and demonstration instructions.

5. Mandatory Data Sources

1. Prepared PCAP files or network-flow datasets containing normal and anomalous traffic.
2. A safe in-house network-traffic simulator generating controlled normal traffic and predefined anomalous scenarios.

The simulator must support at least: Port Scan, DoS-like/connection flood, Brute Force-like behaviour, and Abnormal Connection Pattern. The system must not require access to real networks.

6. Mandatory Data Flow

3. Import PCAP/flow data or generate traffic.
4. Extract and normalise flows.
5. Calculate features.
6. Publish events to the stream-processing layer.
7. Analyse events using time windows.
8. Detect anomalies using rules and a second analytical method.
9. Classify detections and assign severity and confidence.
10. Persist results.
11. Expose results through the API.
12. Present current and historical information in the dashboard.

7. Mandatory Functional Requirements

ID	Area	Requirement
F01	Data ingestion	Import prepared PCAP/flow data and validate input.
F02	Flow extraction	Convert traffic into a normalised flow model.
F03	Feature extraction	Calculate temporal, volume, and connection-related features.
F04	Stream processing	Process data continuously using inter-component communication.
F05	Time windows	Analyse using at least 1 s, 10 s, and 60 s windows.
F06	Rule-based detection	Detect Port Scan, DoS-like, Brute Force-like, and Abnormal Connection Pattern.
F07	Second detection method	Implement a statistical or ML-based method, e.g. z-score,

		moving statistics, Isolation Forest, or equivalent.
F08	Classification	Store type, severity, confidence, timestamp, source, destination, and detection ID.
F09	Explainability	Provide evidence and feature values explaining the alert.
F10	Database	Persist flows, detections, configuration, and historical data.
F11	REST API	Expose current data, history, statistics, and alerts.
F12	Dashboard	Provide traffic monitoring, alerts, statistics, and detection details.
F13	Filtering	Filter history by at least time, type, severity, and source.
F14	Simulator	Generate normal traffic and four predefined anomalous scenarios.
F15	Traceability	Allow an event to be traced from input to the generated detection.

8. Network Flow Model

Each flow must contain at least: start timestamp, duration, source IP, destination IP, source port, destination port, protocol, packet count, byte count, and TCP flag information when available.

The system must additionally calculate packets/sec, bytes/sec, connections per window, unique destination ports, unique destination addresses, and new-connection rate.

9. Anomaly Detection

9.1. Rule-based Detection

The rule-based detector is mandatory and deterministic. Rules must use time windows and measurable features. Parameters must be configurable and default values must be stored in project configuration.

- Port Scan — excessive numbers of different destination ports contacted by one source within a short window.
- DoS-like — unusually high numbers of connections or packets directed at a target.
- Brute Force-like — high numbers of repeated connection attempts to the same target/service.
- Abnormal Connection Pattern — significant deviation from the baseline connection profile.

9.2. Second Detection Method

In addition to rules, the team must implement at least one statistical or machine-learning-based method. It must be described, justified, tested, and compared with the rule-based approach.

9.3. Detection Result

Every detection must contain: unique ID, timestamp, type, severity (LOW/MEDIUM/HIGH/CRITICAL), confidence 0–1, source, destination, time window, key feature values, and a textual explanation.

10. Reference Architecture

The system must contain clearly separated components. The recommended decomposition is:

- Traffic / PCAP Importer
- Flow Extractor
- Traffic Simulator
- Message Broker / Event Transport
- Stream Processor
- Detection Engine
- Alert & Classification Service
- Persistence Layer / Database
- REST API
- Dashboard

The architecture does not have to be microservice-based, but it must provide separation of responsibilities, inter-component communication, and independent testing of key modules. Use of a message broker is mandatory.

11. Non-functional Requirements

ID	Category	Requirement
NFR01	Performance	Process at least 5,000 flows/s for a minimum of 5 minutes.
NFR02	Latency	At 5,000 flows/s, p95 from data acceptance to detection creation must not exceed 3 s in the nominal failure-free scenario.
NFR03	Scalability	At least one processing component must support more than one instance.
NFR04	Resilience	Restart/failure of one processing instance must not stop the entire platform.
NFR05	Reproducibility	The complete system must be deployable using one documented procedure.
NFR06	Security	The API must provide authentication, authorisation for administrative operations, input validation, and basic API-abuse protection.
NFR07	Testability	Key components must have unit and integration tests.

NFR08	Observability	Logs must support tracing data flow and diagnosing failures.
NFR09	Data	No dependency on production data or real networks.
NFR10	Documentation	The repository must contain deployment, configuration, testing, and demonstration instructions.

12. Security Requirements

- Only controlled laboratory data may be used.
- The project must not implement real attacks, security-control bypasses, or scanning outside the test environment.
- User and administrative operations must be separated.
- External input must be validated.
- Secrets and credentials must never be committed to the repository.
- Important errors and administrative events must be logged.

13. Testing and Quality Metrics

- Unit tests — extraction, aggregation, rules, and classification.
- Integration tests — communication, broker, database, and API.
- System tests — complete end-to-end flow.
- Functional tests — every mandatory use case.
- Performance tests — throughput, p95 latency, CPU, and memory.
- Resilience tests — restart, delayed messages, and duplicates.
- Security tests — authentication, authorisation, validation, and API abuse cases.
- Detection quality — precision, recall, F1, false-positive rate, and false-negative rate.

The test report must contain at least: test case, conditions, expected result, actual result, PASS/FAIL status, and comments.

14. Team Organisation

Member	Area	Primary responsibility
1	Network Data & Flow Processing	PCAP/flow ingestion, extraction, and normalisation.
2	Anomaly Detection & Algorithms	Rules, second method, time windows, explainability.
3	Backend & Data Storage	API, database, classification, persistence.
4	Frontend & Visualization	Dashboard, visualisation, filtering, detection details.
5	Infrastructure, Testing & DevOps	Broker, containers, CI/CD, system and performance tests.

15. Schedule — 10 Weeks

Week	Stage	Outcome
1	Problem Definition & Requirements	Vision, requirements, use cases, data, test plan.
2	Architecture & Data Model	Architecture, broker, models, API contract, diagrams.
3	Network Data Ingestion	Importer, flow extraction, normalisation, validation.
4	Traffic Simulator	Normal traffic + four anomalous scenarios + configurable load.
5	Anomaly Detection Engine	Rules, windows, second method, severity, confidence, explainability.
6	Classification, Persistence & API	Classification, database, API, history, filtering.
7	Monitoring Dashboard	Dashboard, charts, alerts, detection details.
8	Integration, Scalability & Resilience	Integration, containers, multiple instances, performance and failure tests.
9	Testing & Security	Full test campaign and quality/security metrics.
10	Finalisation & Final Presentation	Stabilisation, documentation, final test and demonstration.

16. Assessment Components

Component	Assessment scope
1. Vision Document + Test Plan	Requirements, scope, use cases, and testing strategy.
2. Midterm Presentation + Architecture Document	Architecture, data flow, data model, technology decisions, algorithms.
3. Application / Platform	Completeness, integration, implementation quality, UX, end-to-end operation.
4. Testing + Test Report	Coverage, results, performance, detection quality, resilience, security.
5. Final Presentation	Working demo, architecture, algorithms, results, and limitations.

17. Vision Document — Required Content

- Introduction and document purpose.
- Problem and context.
- System goal and expected benefits.
- Users and stakeholders.
- Use cases.
- Product function overview.

- Functional and non-functional requirements.
- Assumptions, constraints, and dependencies.
- Test data and demonstration environment.

18. Application Architecture Document — Required Content

- High-level architecture.
- Components and responsibilities.
- Communication and message broker.
- Data-flow model.
- Data/database model.
- API contracts.
- Detection algorithms and time windows.
- Technology decision rationale.
- Scalability and resilience.
- Architecture, data-flow, sequence, and data-model diagrams.

19. Test Plan and Test Report

- Scope, objectives, and strategy.
- Test environment and data.
- Responsibilities.
- Entry and exit criteria.
- Scenarios and test cases.
- Unit, integration, system, functional, and non-functional tests.
- Performance and resilience.
- Security.
- Precision, recall, F1, false-positive and false-negative rate.
- Result table: test / expected / actual / status / comment.
- Known limitations and defects.

20. Project Completion Criteria

13. F01–F15 implemented and demonstrated.
14. NFR01–NFR10 verified through tests.
15. End-to-end operation from input to dashboard.
16. All four required anomalous scenarios detected.
17. Both detection methods operational.
18. Results include explainability, severity, and confidence.
19. NFR01–NFR02 satisfied.
20. Complete project documentation.
21. Deployment and demonstration instructions in the repository.
22. Final presentation includes working demonstration and test results.

21. Explicitly Out of Scope

- Integration with production networks.
- Processing real end-user traffic.
- Active traffic blocking, attack execution, or vulnerability exploitation.
- Building a deep-learning model from scratch.
- Mandatory use of a public cloud.
- A mandated programming language, framework, or database vendor; the team chooses and justifies the technology.

22. Final Deliverable

The final deliverable is a complete, deployable, and tested Network Anomaly Detection Platform that accepts controlled network-traffic data, processes it as a stream, detects and classifies anomalies, persists results, exposes them through an API, and presents them in a dashboard. The project demonstrates competencies in system design, software engineering, data processing, algorithms, distributed systems, testing, security, and DevOps.